

Working with Kubernetes Nodes

Course Lab Project | DevOps & Cloud Engineering

Tools: Minikube · kubectl · Ubuntu (VMware) · Kubernetes Author: Oluwaseun Osunsola LinkedIn: [See Profile](#)

Table of Contents

- Working with Kubernetes Nodes
 - Table of Contents
 - Project Overview
 - What Is a Kubernetes Node?
 - Prerequisites
 - Lab Implementation
 - Step 1 — Verify Minikube Installation
 - Step 2 — Verify kubectl Installation
 - Step 3 — Start the Minikube Cluster
 - Step 4 — View Nodes in the Cluster
 - Step 5 — View Nodes (Extended / Wide View)
 - Step 6 — Inspect the Node with `describe`
 - Step 7 — Observe Conditions, Capacity, Allocatable, System Info & Address
 - Step 8 — Observe Non-Terminated Pods and Events
 - Step 9 — Stop the Minikube Cluster
 - Step 10 — Confirm Error After Cluster Stop
 - Step 11 — Restart the Minikube Cluster
 - Step 12 — Prove State Persistence (AGE Unchanged)
 - Step 13 — Delete the Minikube Cluster
 - Key Concepts Demonstrated
 - Commands Quick Reference

Project Overview

This project demonstrates the management of Kubernetes nodes using a local Minikube cluster running on Ubuntu inside a VMware environment. The lab covers the full lifecycle of a Minikube cluster — from verification and startup through node inspection, graceful stopping, state persistence, and final deletion — as required by the course instructor.

Environment:

Component	Detail
Host OS	Ubuntu (VMware VM)
Kubernetes Distribution	Minikube (single-node)
CLI Tool	kubectl

Component	Detail
Node Type	Control plane + worker (combined)

What Is a Kubernetes Node?

In Kubernetes, a **node** is a physical or virtual machine that runs the Kubernetes software and serves as a worker machine in the cluster. Think of a node as a dedicated employee in an office — responsible for executing tasks and hosting containers to ensure seamless application performance.

Nodes are responsible for running **Pods**, which are the basic deployable units in Kubernetes. Each node in a Kubernetes cluster typically represents a single host system and carries the following core components:

- **kubelet** — agent that communicates with the control plane
- **kube-proxy** — handles network routing for services
- **Container runtime** — runs the actual containers (e.g., Docker, containerd)

In a Minikube setup, a single node acts as both the control plane and the worker, making it ideal for local development and testing without the overhead of a full production cluster.

Prerequisites

Before beginning the lab, both Minikube and kubectl must be installed and accessible from the terminal.

```
# Check Minikube
minikube version

# Check kubectl
kubectl version --client
```

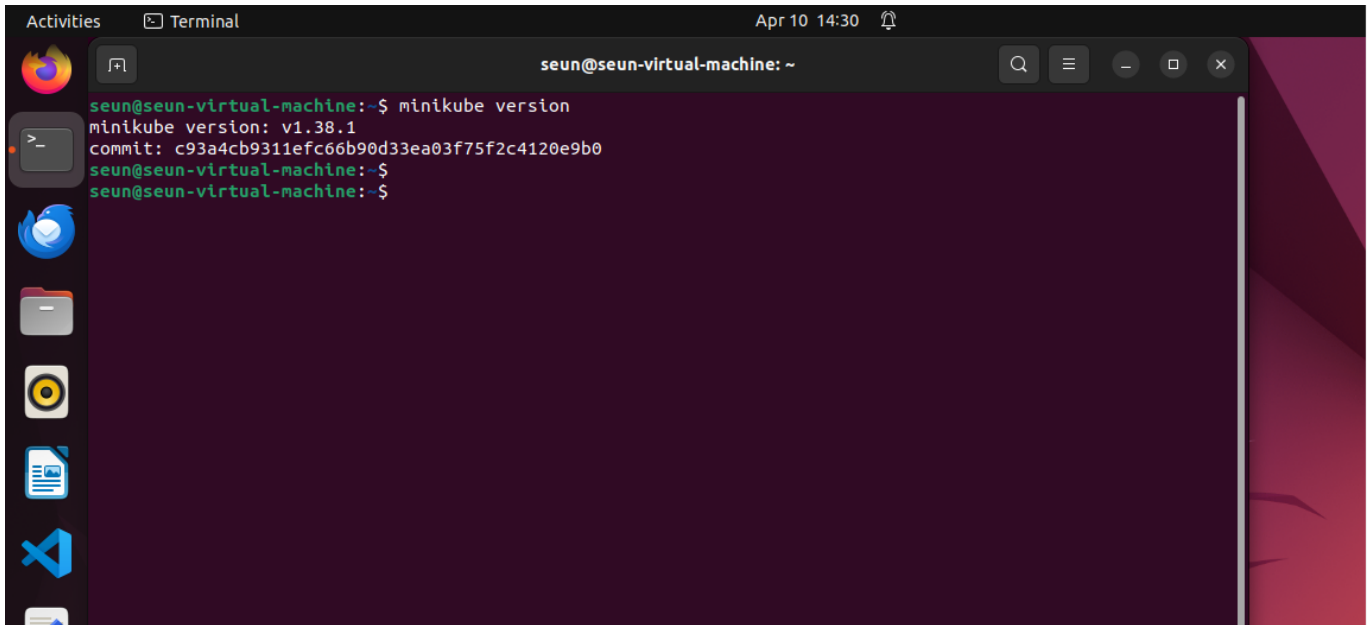
Lab Implementation

Step 1 — Verify Minikube Installation

The first step confirms that the **minikube** binary is installed and accessible in the system path. This ensures the environment is correctly set up before attempting to start any cluster.

```
minikube version
```

Expected output: Minikube version string confirming it is installed.

A terminal window titled 'seun@seun-virtual-machine: ~' showing the output of the 'minikube version' command. The output displays the minikube version as v1.38.1 and the commit hash as c93a4cb9311efc66b90d33ea03f75f2c4120e9b0. The terminal window is part of a desktop environment with a sidebar showing various application icons.

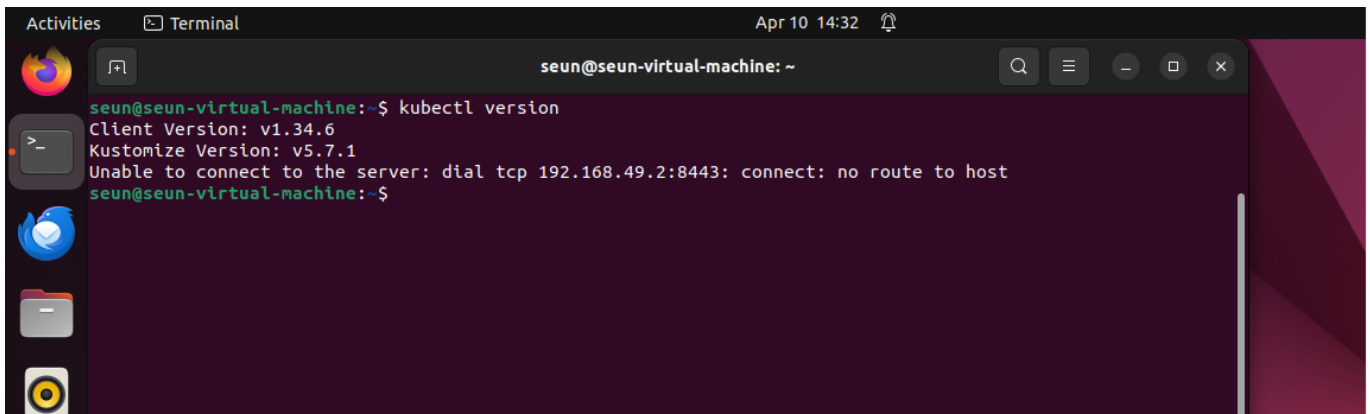
```
seun@seun-virtual-machine:~$ minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
seun@seun-virtual-machine:~$
seun@seun-virtual-machine:~$
```

Step 2 — Verify kubectl Installation

kubectl is the command-line tool used to interact with the Kubernetes API. This step confirms it is installed and shows the client version.

```
kubectl version --client
```

Expected output: kubectl client version details.

A terminal window titled 'seun@seun-virtual-machine: ~' showing the output of the 'kubectl version' command. The output displays the Client Version as v1.34.6 and the Kustomize Version as v5.7.1. It also shows an error message: 'Unable to connect to the server: dial tcp 192.168.49.2:8443: connect: no route to host'. The terminal window is part of a desktop environment with a sidebar showing various application icons.

```
seun@seun-virtual-machine:~$ kubectl version
Client Version: v1.34.6
Kustomize Version: v5.7.1
Unable to connect to the server: dial tcp 192.168.49.2:8443: connect: no route to host
seun@seun-virtual-machine:~$
```

Step 3 — Start the Minikube Cluster

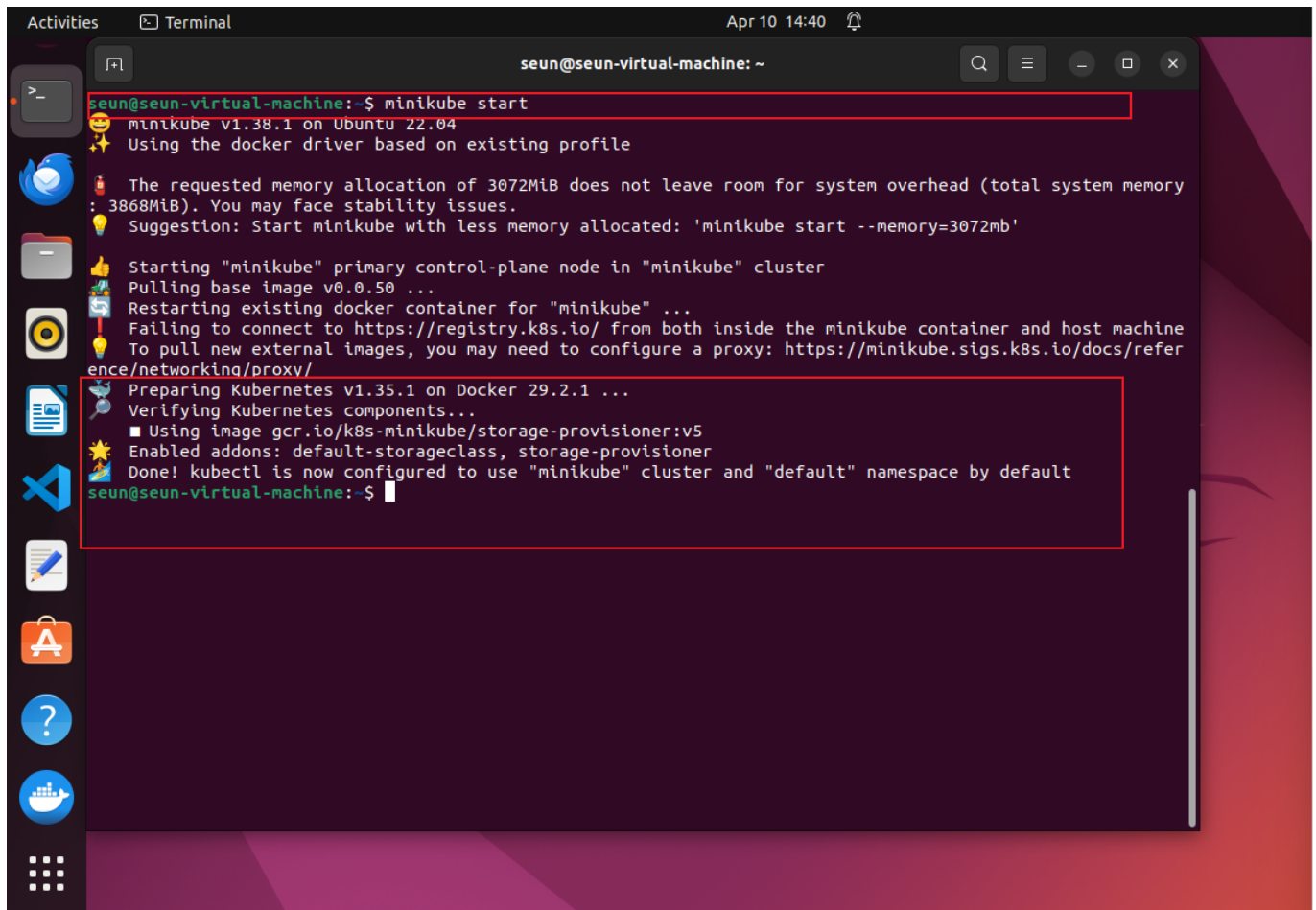
This command provisions a local single-node Kubernetes cluster. Minikube creates a virtual machine (or container, depending on the driver) and configures **kubectl** to point to it automatically. On first run, this may take a few minutes as images are downloaded.

```
minikube start
```

Expected output:

```
* minikube v1.x.x on Ubuntu xx.xx
* Starting control plane node minikube ...
* Done! kubectl is now configured to use "minikube"
```

The key confirmation line is: **kubectl is now configured to use "minikube"** — this tells us kubectl is now wired to the local cluster.

**Step 4 — View Nodes in the Cluster**

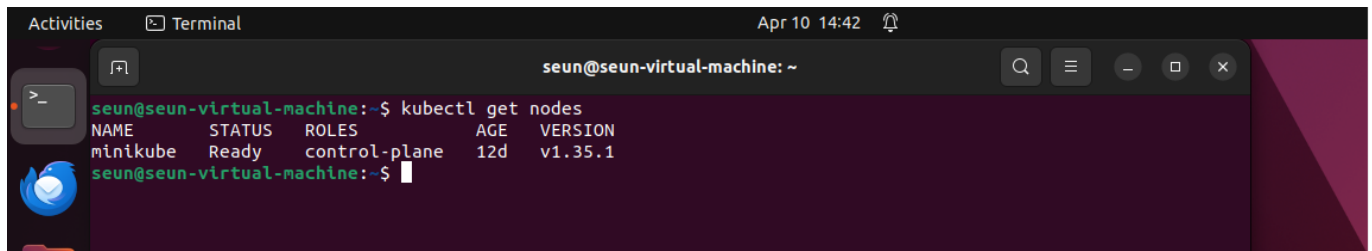
Lists all nodes in the cluster and displays their current status. This is the primary command for verifying that the cluster has an active, healthy node.

```
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	Xm	v1.29.x

The **Ready** status confirms the node is healthy and able to accept workloads. The **control-plane** role indicates this node also manages cluster state — typical for single-node Minikube setups.



```
seun@seun-virtual-machine:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane 12d   v1.35.1
seun@seun-virtual-machine:~$
```

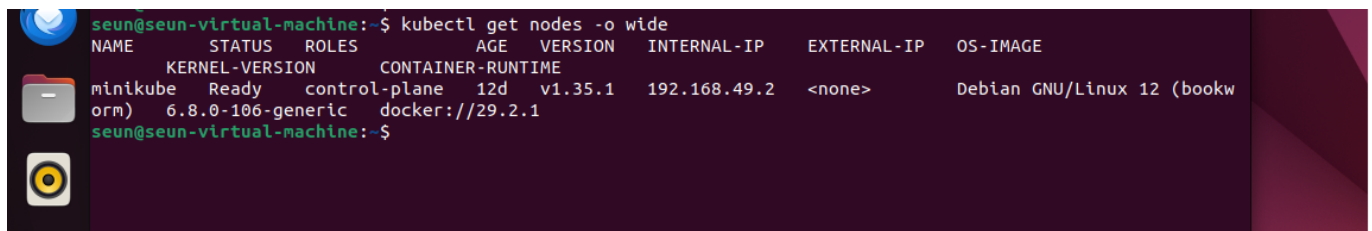
Step 5 — View Nodes (Extended / Wide View)

The **-o wide** flag extends the output with additional details including the internal IP address, external IP, the OS image, the kernel version, and the container runtime. This provides a more complete picture of the node's environment.

```
kubectl get nodes -o wide
```

Additional columns revealed:

- **INTERNAL-IP** — the node's IP within the cluster network
- **OS-IMAGE** — the operating system running on the node
- **KERNEL-VERSION** — the Linux kernel version
- **CONTAINER-RUNTIME** — shows the runtime (e.g., docker, containerd)



```
seun@seun-virtual-machine:~$ kubectl get nodes -o wide
NAME        STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
minikube    Ready     control-plane 12d   v1.35.1   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)
6.8.0-106-generic docker://29.2.1
seun@seun-virtual-machine:~$
```

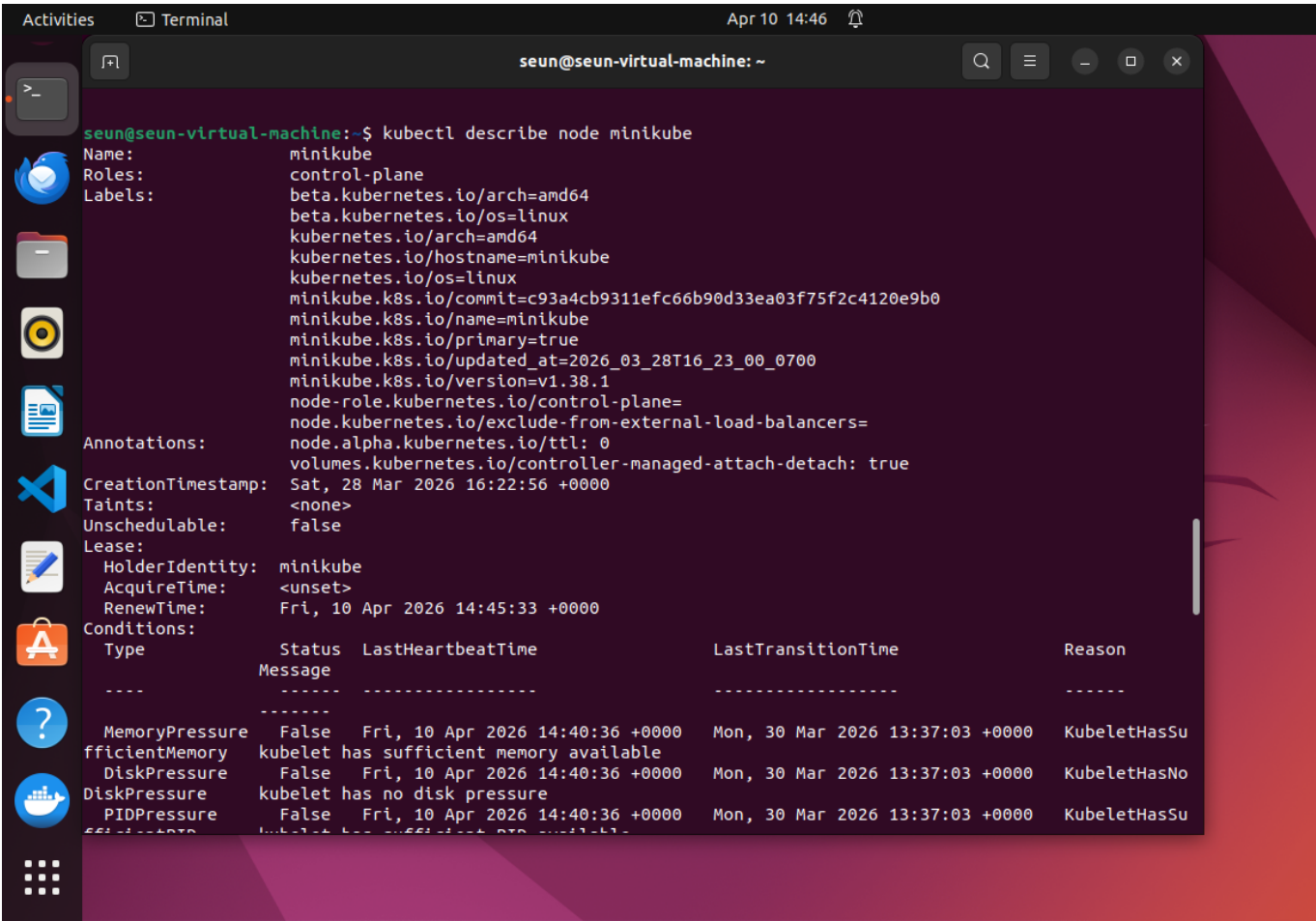
Step 6 — Inspect the Node with **describe**

The **kubectl describe node** command provides a comprehensive, human-readable output of everything Kubernetes knows about a specific node. Replace **<node-name>** with the actual name shown in **kubectl get nodes** — in Minikube this will be **minikube**.

```
kubectl describe node minikube
```

This command outputs several key sections which are examined in the following steps. To scroll through the output comfortably:

```
kubectl describe node minikube | less
```



Step 7 — Observe Conditions, Capacity, Allocatable, System Info & Address

Within the `kubectl describe node` output, this step focuses on four critical sections that reveal the node's health, resource limits, and system environment:

Section	What It Shows
Addresses	Internal IP, hostname of the node
Conditions	Health checks — Ready, MemoryPressure, DiskPressure, PIDPressure
Capacity	Total resources available on the node (CPU, memory, pods)
Allocatable	Resources available for scheduling pods (after system reservations)
System Info	OS, kernel, container runtime, kubelet/kube-proxy versions

The **Conditions** section is particularly important — all conditions except `Ready` should show `False`, while `Ready` should show `True`, confirming a healthy node.

```

RenewTime:      Fri, 10 Apr 2026 14:45:33 +0000
Conditions:
  Type            Status    LastHeartbeatTime             LastTransitionTime             Reason                       Message
  ----            -
MemoryPressure    False     Fri, 10 Apr 2026 14:40:36 +0000 Mon, 30 Mar 2026 13:37:03 +0000 KubeletHasSufficientMemory    kubelet
has sufficient memory available
DiskPressure      False     Fri, 10 Apr 2026 14:40:36 +0000 Mon, 30 Mar 2026 13:37:03 +0000 KubeletHasNoDiskPressure      kubelet
has no disk pressure
PIDPressure       False     Fri, 10 Apr 2026 14:40:36 +0000 Mon, 30 Mar 2026 13:37:03 +0000 KubeletHasSufficientPID       kubelet
has sufficient PID available
Ready            True      Fri, 10 Apr 2026 14:40:36 +0000 Mon, 30 Mar 2026 13:56:28 +0000 KubeletReady                   kubelet
is posting ready status
Addresses:
  InternalIP: 192.168.49.2
  Hostname: minikube
Capacity:
  cpu:                2
  ephemeral-storage: 40585548Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             3961812Ki
  pods:               110
Allocatable:
  cpu:                2
  ephemeral-storage: 40585548Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             3961812Ki
  pods:               110
System Info:
  Machine ID:         86e0bfeb3f77427722393c2969964edb
  System UUID:        7375b539-0acb-4049-9153-8a6d3e5ce381
  Boot ID:            c78f6a43-a979-4307-b0dd-955a802a7a2e
  Kernel Version:     6.8.0-106-generic
  OS Image:           Debian GNU/Linux 12 (bookworm)
  Operating System:   linux
  Architecture:       amd64
  Container Runtime Version: docker://29.2.1
  Kubelet Version:    v1.35.1
  Kube-Proxy Version: v1.35.1

```

Step 8 — Observe Non-Terminated Pods and Events

The `kubectl describe node` output also contains two sections relevant to workloads and node history:

- **Non-terminated Pods** — lists all pods currently running or scheduled on this node, along with their namespace, CPU requests, and memory limits. Even in a fresh Minikube cluster, several system pods (such as `kube-apiserver`, `coredns`, and `kube-proxy`) will appear here.
- **Events** — shows recent node-level events, such as when the node was registered, when the kubelet started, or any warnings about resource pressure. This is a key debugging section in production environments.

```
# These sections appear at the bottom of:
kubectl describe node minikube
```

```

Machine ID: 86e0bfefb3f77427722393c2969964edb
System UUID: 7375b539-0acb-4049-9153-8a6d3e5ce381
Boot ID: c78f6a43-a979-4307-b0dd-955a802a7a2e
Kernel Version: 6.8.0-106-generic
OS Image: Debian GNU/Linux 12 (bookworm)
Operating System: linux
Architecture: amd64
Container Runtime Version: docker://29.2.1
Kubelet Version: v1.35.1
Kube-Proxy Version:
PodCIDR: 10.244.0.0/24
PodCIDRs: 10.244.0.0/24
Non-terminated Pods: (11 in total)
Namespace      Name                                CPU Requests  CPU Limits    Memory Requests  Memory Limits  Age
-----
default        my-nginx-575c69474d-5d7tt          0 (0%)        0 (0%)        0 (0%)           0 (0%)         12d
default        my-nginx-575c69474d-5kndg          0 (0%)        0 (0%)        0 (0%)           0 (0%)         12d
default        my-nginx-575c69474d-m8w8X          0 (0%)        0 (0%)        0 (0%)           0 (0%)         12d
demo           nginx                               0 (0%)        0 (0%)        0 (0%)           0 (0%)         12d
kube-system    coredns-7d764666f9-bzlbl          100m (5%)     0 (0%)        70Mi (1%)        170Mi (4%)     12d
kube-system    etcd-minikube                     100m (5%)     0 (0%)        100Mi (2%)       0 (0%)         12d
kube-system    kube-apiserver-minikube            250m (12%)    0 (0%)        0 (0%)           0 (0%)         12d
kube-system    kube-controller-manager-minikube    200m (10%)    0 (0%)        0 (0%)           0 (0%)         12d
kube-system    kube-proxy-fkqbk                   0 (0%)        0 (0%)        0 (0%)           0 (0%)         12d
kube-system    kube-scheduler-minikube            100m (5%)     0 (0%)        0 (0%)           0 (0%)         12d
kube-system    storage-provisioner                0 (0%)        0 (0%)        0 (0%)           0 (0%)         12d
Allocated resources:
(Total limits may be over 100 percent, i.e., overcommitted.)
Resource      Requests    Limits
-----
cpu            750m (37%)  0 (0%)
memory         170Mi (4%)  170Mi (4%)
ephemeral-storage  0 (0%)     0 (0%)
hugepages-1Gi    0 (0%)     0 (0%)
hugepages-2Mi    0 (0%)     0 (0%)
Events:
Type      Reason      Age   From      Message
-----
Normal    RegisteredNode  8m8s  node-controller  Node minikube event: Registered Node minikube in Controller
seun@seun-virtual-machine:~$

```

Step 9 — Stop the Minikube Cluster

Stops the running Minikube cluster while **preserving its state**. This is equivalent to shutting down a machine without deleting it — all configurations, deployments, and resources are saved and will be available when the cluster is started again.

```
minikube stop
```

Expected output:

```
* Stopping node "minikube" ...
* 1 node stopped.
```

Key distinction: `minikube stop` $\neq$ `minikube delete`. Stop is reversible; delete is not.

```

seun@seun-virtual-machine:~$ minikube stop
Stopping node "minikube" ...
Powering off "minikube" via SSH ...
1 node stopped.
seun@seun-virtual-machine:~$

```

Step 10 — Confirm Error After Cluster Stop

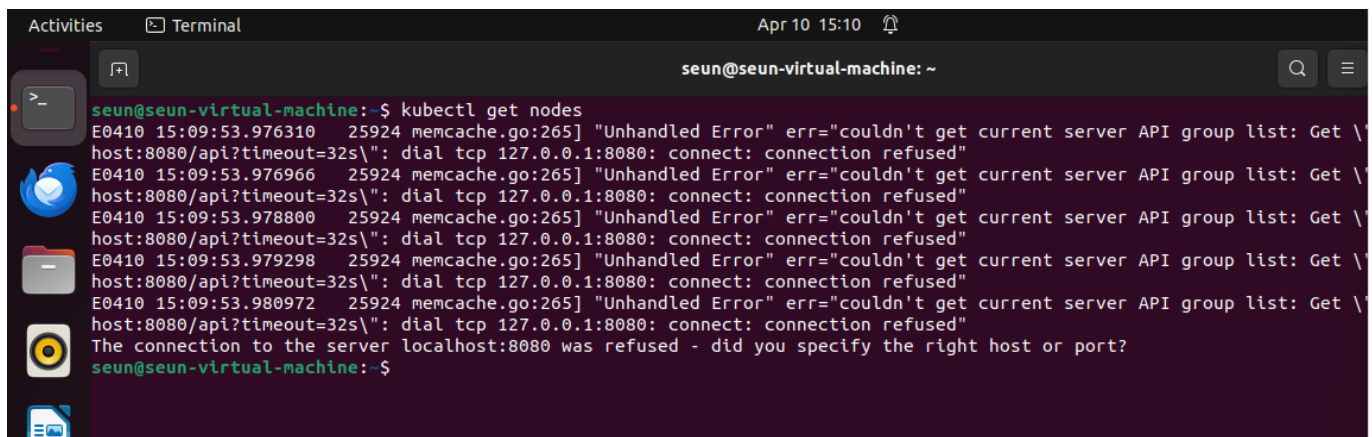
After stopping the cluster, running `kubectl get nodes` demonstrates that the cluster is no longer accessible. This confirms that Minikube must be running for `kubectl` to communicate with the Kubernetes API.

```
kubectl get nodes
```

Expected output: A connection error or timeout, such as:

```
The connection to the server 192.168.x.x:8443 was refused
```

This error is expected and intended — it proves the cluster is stopped and the API server is no longer listening.



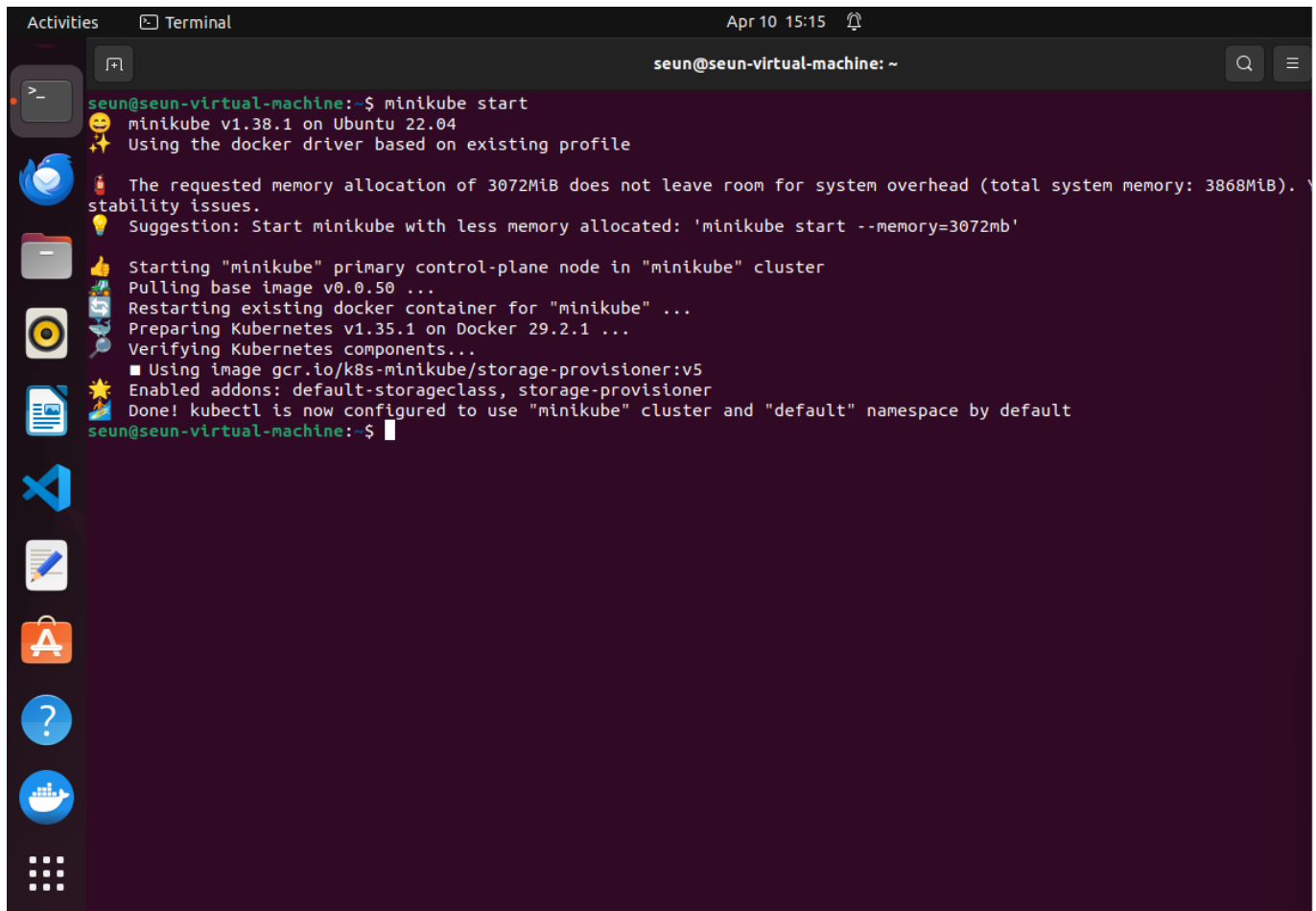
```
seun@seun-virtual-machine:~$ kubectl get nodes
E0410 15:09:53.976310 25924 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \
host:8080/api?timeout=32s\": dial tcp 127.0.0.1:8080: connect: connection refused"
E0410 15:09:53.976966 25924 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \
host:8080/api?timeout=32s\": dial tcp 127.0.0.1:8080: connect: connection refused"
E0410 15:09:53.978800 25924 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \
host:8080/api?timeout=32s\": dial tcp 127.0.0.1:8080: connect: connection refused"
E0410 15:09:53.979298 25924 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \
host:8080/api?timeout=32s\": dial tcp 127.0.0.1:8080: connect: connection refused"
E0410 15:09:53.980972 25924 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \
host:8080/api?timeout=32s\": dial tcp 127.0.0.1:8080: connect: connection refused"
The connection to the server localhost:8080 was refused - did you specify the right host or port?
seun@seun-virtual-machine:~$
```

Step 11 — Restart the Minikube Cluster

Restarts the previously stopped cluster. Because `minikube stop` preserves state, this command resumes the cluster rather than creating a new one.

```
minikube start
```

This is faster than the initial start since no new images need to be downloaded.



```
seun@seun-virtual-machine:~$ minikube start
minikube v1.38.1 on Ubuntu 22.04
Using the docker driver based on existing profile

The requested memory allocation of 3072MiB does not leave room for system overhead (total system memory: 3868MiB).
stability issues.
Suggestion: Start minikube with less memory allocated: 'minikube start --memory=3072mb'

Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.50 ...
Restarting existing docker container for "minikube" ...
Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
Verifying Kubernetes components...
  Using image gcr.io/k8s-minikube/storage-provisioner:v5
Enabled addons: default-storageclass, storage-provisioner
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
seun@seun-virtual-machine:~$
```

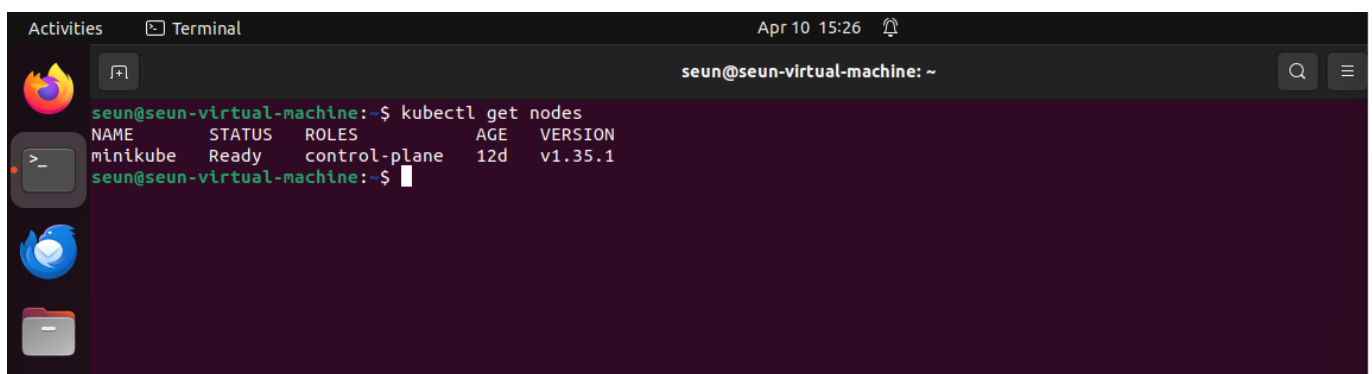
Step 12 — Prove State Persistence (AGE Unchanged)

After restarting, running `kubectl get nodes` again confirms that:

1. The node returns to `Ready` status
2. **The `AGE` value continues from where it was** — it does not reset to `0m`

The persistent `AGE` value is the proof that `minikube stop` preserved the cluster state rather than creating a new one from scratch. This is a key conceptual distinction between `stop` and `delete`.

```
kubectl get nodes
```



```
seun@seun-virtual-machine:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane   12d   v1.35.1
seun@seun-virtual-machine:~$
```

Step 13 — Delete the Minikube Cluster

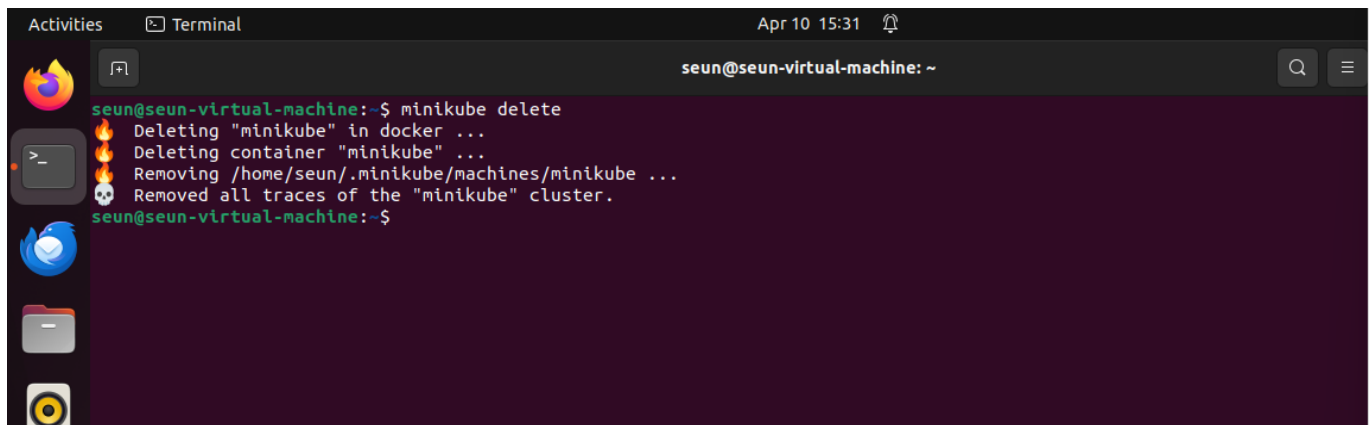
Permanently deletes the Minikube cluster and all associated resources including VMs, containers, volumes, and network configurations. Unlike `stop`, this action cannot be undone and the `AGE` will reset to zero if the cluster is re-created.

```
minikube delete
```

Expected output:

```
* Deleting "minikube" in docker ...
* Removed all traces of the "minikube" cluster.
```

⚠ Always capture required screenshots **before** running this command.

A screenshot of a terminal window titled 'Terminal' with a dark background. The prompt is 'seun@seun-virtual-machine: ~'. The command 'minikube delete' has been executed, resulting in the following output: 'Deleting "minikube" in docker ...', 'Deleting container "minikube" ...', 'Removing /home/seun/.minikube/machines/minikube ...', and 'Removed all traces of the "minikube" cluster.'. The prompt returns to 'seun@seun-virtual-machine:~\$'. On the left side of the terminal window, there is a vertical dock with icons for a file manager, a terminal, and other applications.

Key Concepts Demonstrated

Node lifecycle management — the full flow from `start` → inspect → `stop` → restart → `delete` was exercised, covering all three cluster management commands from the instructor's requirements.

State preservation vs. deletion — Steps 9–12 explicitly demonstrate the difference between `minikube stop` (preserves state, `AGE` persists) and `minikube delete` (destroys everything). This is a critical operational concept when managing development clusters.

Node inspection depth — `kubectl describe node` was used to examine all five key sections: Conditions, Capacity, Allocatable, System Info, and Non-Terminated Pods. Understanding these sections is essential for diagnosing node health in any Kubernetes environment.

Single-node cluster architecture — Minikube runs the control plane and worker roles on the same node, which is appropriate for local development but differs from production setups where these roles are separated across multiple machines.

Commands Quick Reference

Command	Purpose
<code>minikube version</code>	Verify Minikube is installed
<code>kubectl version --client</code>	Verify kubectl is installed
<code>minikube start</code>	Start (or resume) the local cluster
<code>minikube stop</code>	Stop the cluster, preserve state
<code>minikube delete</code>	Delete the cluster and all resources
<code>kubectl get nodes</code>	List all nodes and their status
<code>kubectl get nodes -o wide</code>	List nodes with extended system details
<code>kubectl describe node <name></code>	Full inspection of a specific node
<code>kubectl describe node minikube \ less</code>	Scroll through describe output

Lab completed on Ubuntu (VMware VM) using Minikube single-node Kubernetes cluster.